Sri Lanka Institute of Information Technology



SE2030 - Software Engineering

Year 2 Semester 1- 2026

Final Report

Web-based Apartment Sales System

Group 2026-Y2-S1-MLB-B12G1-01

IT Number	Name
IT25101419	Jayasuriya J.A.P.S
IT25102211	Mannapperuma M.J.P
IT25103164	Bandara A.M.P.L
IT25100456	Bandara R.H.D.T
IT25102571	Wijethunga S.M
IT25100040	Shankeerthan S

Table of Contents

1. Introduction 3-4
   - 1.1 Project Overview 3
   - 1.2 Objectives 3
   - 1.3 Target Users and Stakeholders 3
   - 1.4 Scope & Limitations 4
2. Requirements 5-6
   - 2.1 Functional Requirements 5
   - 2.2 Non-Functional Requirements 5
   - 2.3 Constraints & Assumptions 6
3. Design 7-12
   - 3.1 Use Case Diagram 7
   - 3.2 EER Diagram 8
   - 3.3 UI Sketches 9-10
   - 3.4 Design Patterns Applied 11-12
4. Implementation 13-16
   - 4.1 Tools & Technologies Used 13
   - 4.2 System Architecture 13-14
   - 4.3 Key Features Developed 14
   - 4.4 Screenshots of Key Functions 15-16
5. Project Management 17-18
   - 5.1 Agile Approach 17
   - 5.2 Final Sprint Summary 17
   - 5.3 Task Distribution 18
   - 5.4 Development Milestones 18
6. Conclusion & Future Works 19
   - 6.1 Summary of Achievements 19
   - 6.2 Key Challenges Addressed 19
   - 6.3 Future Improvements 19
7. Contribution & Lessons Learned 20
8. References 21

1. Introduction

1.1 Project Overview

The Web-based Apartment Sales System is a centralized Java/Spring Boot web application with an SQL database designed to improve apartment sales operations. It replaces fragmented spreadsheets, paper records, and disconnected communication with a single system for customer and lead management, apartment inventory, booking and reservations, payments and installment schedules, promotions, agent performance, and management reporting.

The system addresses common operational problems such as duplicate records, delayed follow-up, inaccurate availability information, difficulty tracing payments, and time-consuming report preparation. By maintaining shared and role-based data, the platform is intended to increase operational accuracy, reduce manual effort, and provide clearer buying experience for apartment customers.

1.2 Objectives

- Automate and streamline core apartment sales processes.
- Maintain accurate and current apartment inventory and availability information.
- Support lead capture, follow-up, customer communication, and sales pipeline monitoring.
- Enable apartment search, booking, reservations, and status management while reducing double allocation.
- Track customer payments, installment schedules, balances, and receipts.
- Manage promotions and special offers.
- Monitor sales activity, agent performance, and management-level business indicators.
- Reduce manual paperwork and data-entry errors through structured workflows and validation.
- Improve transparency and service quality for apartment buyers and staff.

1.3 Target Users and Stakeholders

Stakeholder	Primary interaction with the system
Sales Manager	Monitors sales activity, targets, agent performance, and business reports.
Apartment Buyer	Searches apartments, views details/offers, reserves units, and tracks payments/purchase status.
Sales Agent	Records and follows up leads, manages assigned work, and interacts with buyers.
Customer Relations Officer	Handles inquiries, communication history, reservation-related support, and customer information.
Finance & Payments Officer	Records payments, manage installments, balances, receipts, and finance reports.
Property Development Manager	Maintains projects, unit pricing, availability, and status.
Operations Director	Reviews business performance, sales reports, apartment availability, agent performance, and booking statistics.
System Administrator	Supports user access, security, and system administration.

1.4 Scope & Limitations

The project scope covers a web-based solution for apartment sales management in the Sri Lankan context. It includes internal sales operations, apartment inventory and project information, leads and customer records, reservations, payments and installments, promotions, and management reporting. The system is intended for use through a web browser and does not include a dedicated native mobile application.

- Requires a stable internet connection and does not provide offline operation.
- No dedicated native mobile application is included.
- Advanced third-party notification integrations such as WhatsApp or full push-notification services are outside the current scope.
- Data quality depends on users entering accurate and complete information.
- The solution focuses on apartment sales; rental and property-maintenance workflows are outside scope.
- The present implementation is primarily intended for a single organization rather than very large enterprise-scale deployment.

2. Requirements

2.4 Functional Requirements

ID / Requirement	Description
Lead Management	Capture, organize, assign, update, and follow customer leads through the sales pipeline while retaining relevant follow-up information.
Apartment Inventory Management	Maintain apartment project and unit information including pricing, specifications, availability, and status.
Booking and Reservation Management	Allow eligible users to reserve available units, manage booking details, and prevent duplicate allocation.
Payment and Installment Tracking	Record payments, maintain installment schedules, track balances and due dates, and generate payment records or receipts.
Promotions and Special Offers	Create, update, activate/deactivate, and present promotional offers to relevant users.
Agent Portfolio and Performance Management	Associate external property listings with the responsible agent. Allow an agent to create, edit and remove permitted listings.
Customer Relationship Support	Maintain customer details, inquiries, communication history, and support information.
Search and Availability	Allow users to search/filter apartment information and identify available, reserved, or sold units.
Notifications and Communication	Provide basic alerts or reminders for bookings, payments, promotions, and important events.
Reporting and Business Monitoring	Present dashboards and reports for sales, bookings, finance, apartment availability, and operational performance.

2.5 Non-Functional Requirements

Category	Requirement
Usability	Interfaces should be clear, consistent, role-appropriate, and easy to navigate.
Performance	Common operations such as login, search, record retrieval, and updates should respond promptly under normal project usage.
Security	Authentication and role-based access should protect customer, financial, and operational data; passwords and sensitive data must be handled securely.
Reliability	Database operations should preserve data consistency and avoid duplicating or conflicting records.
Availability	The system should remain available during expected working hours subject to internet and hosting availability.
Responsiveness	Web pages should adapt to desktop, tablet, and mobile screen sizes.
Maintainability	The codebase should separate concerns into manageable components and use clear naming and modular structure.

Data integrity	Required fields, formats, business rules, and status transitions should be validated before data is stored.

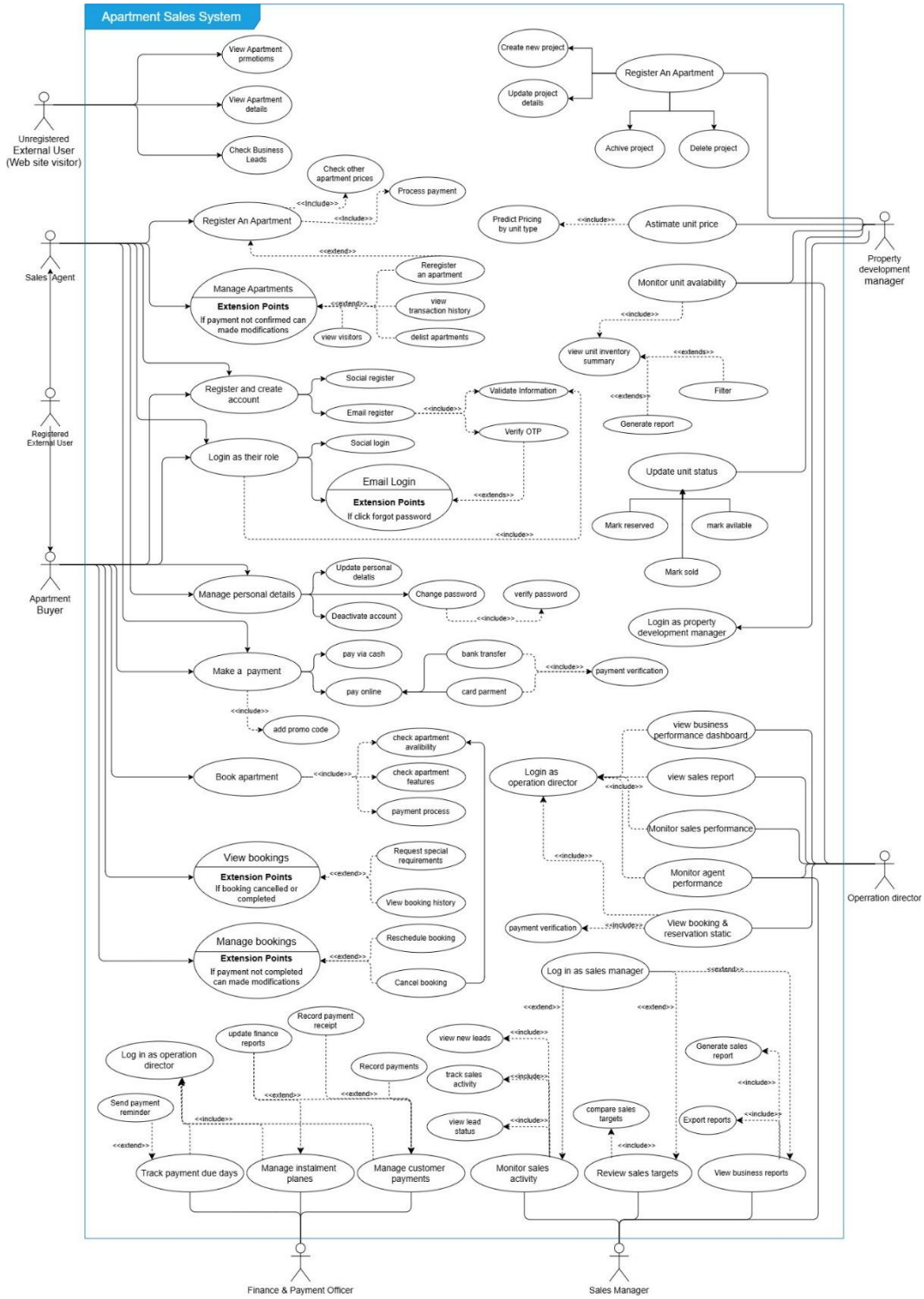
2.6 Constraints & Assumptions

- The system is implemented as a Java/Spring Boot web application backed by an SQL database.
- Users are assumed to have internet access and a supported web browser.
- Role permissions are assumed to be correctly configured for staff and customer accounts.
- Apartment pricing, availability, payment, and customer data depend on authorized users maintaining accurate records.
- External integrations beyond the implemented project scope are not assumed to be available.

3. Design

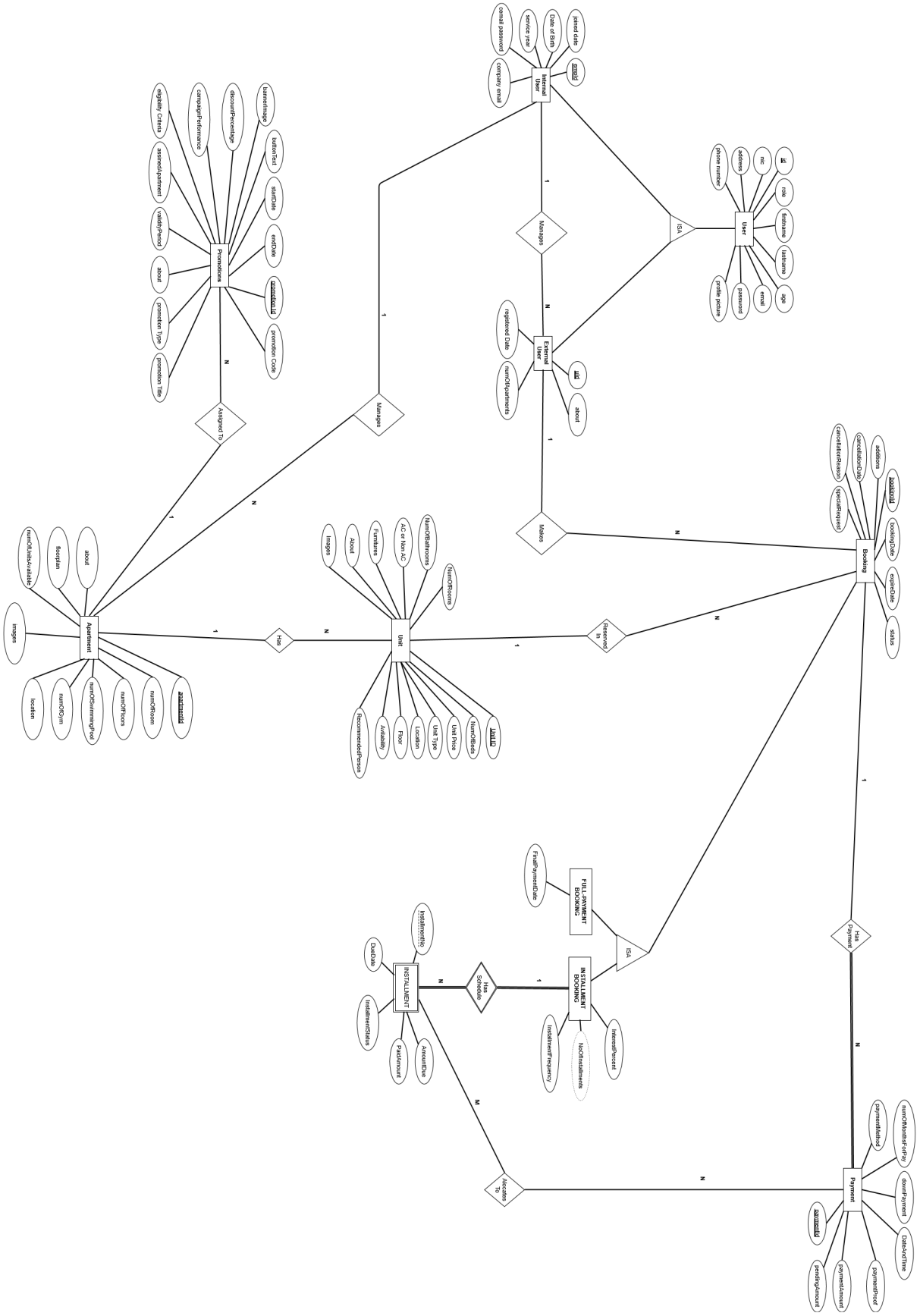
3.1 Use Case Diagram

The final use case model identifies the major external and internal actors and connects them to the functions they perform. Common account functions are shared by registered users, while role-specific functions are separated for buyers, sales staff, property management, finance, sales management, and the Operations Director. Include/extend relationships are used where one behavior forms a required or conditional part of another workflow.



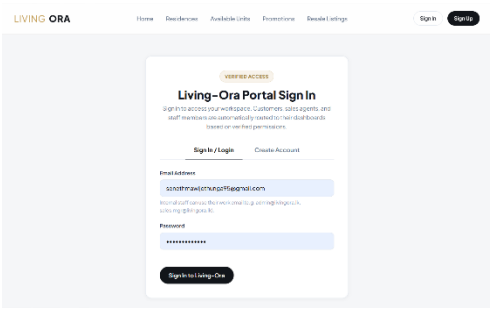
3.2 EER Diagram

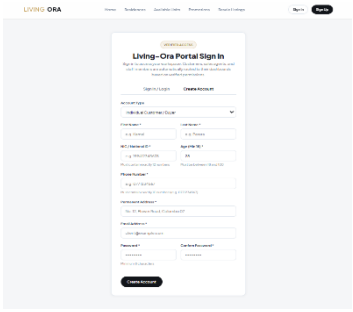
The database model structures the main business data around users, apartments/projects, reservations/bookings, leads, promotions, and payment/installment information. Relationships are designed to preserve traceability between buyers, apartment units, reservations, financial transactions, and management records.



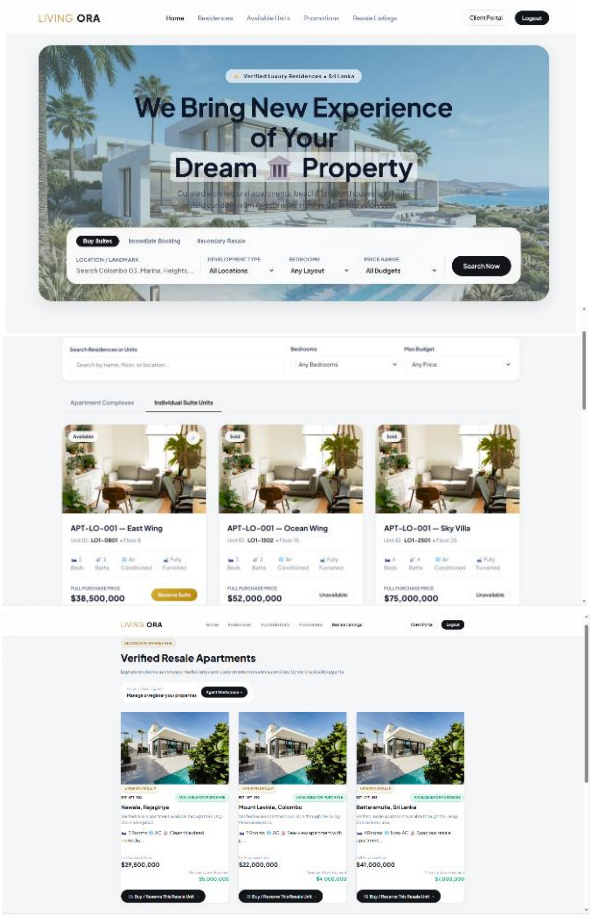
3.3 UI Sketches

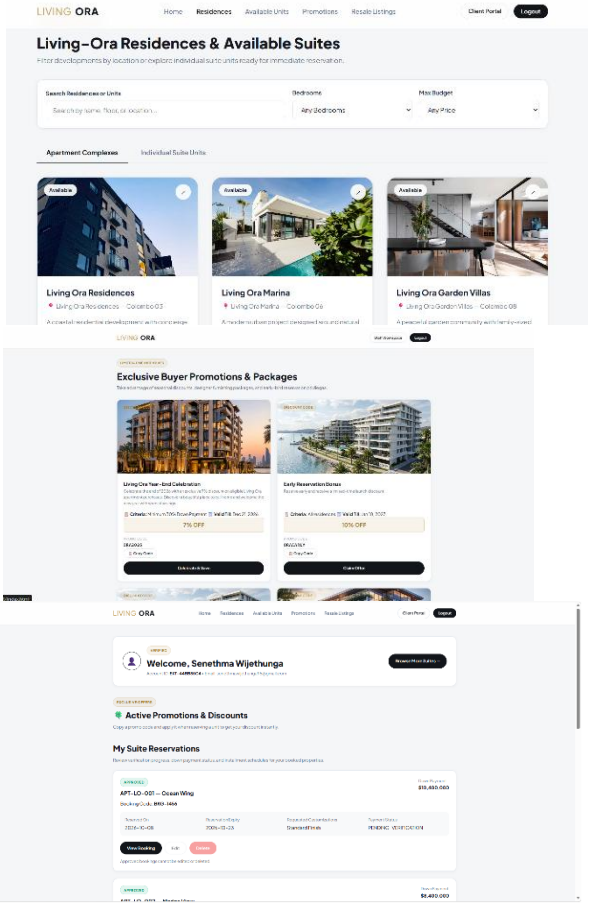
Login/Register



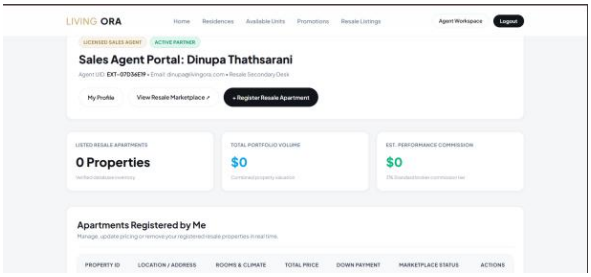


External User UI

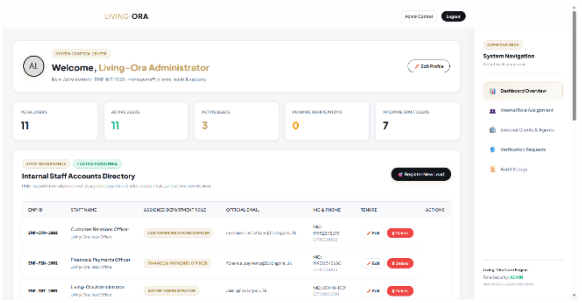


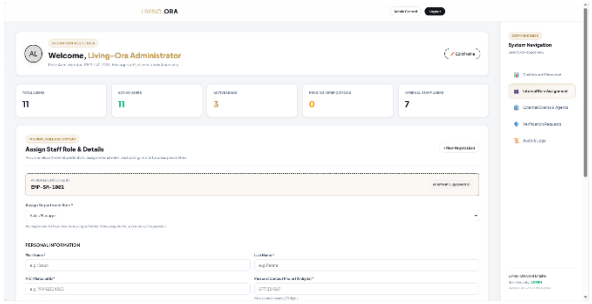


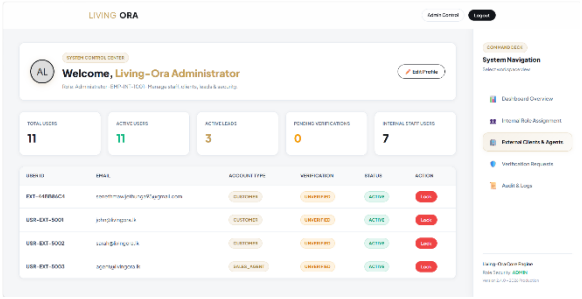
External Agent UI

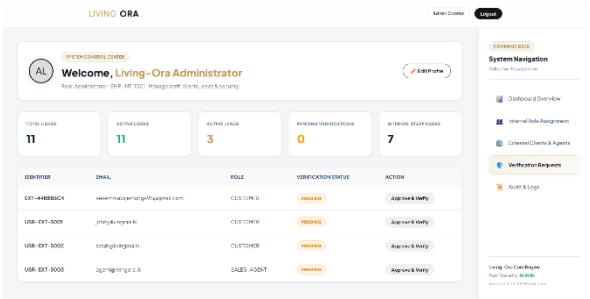


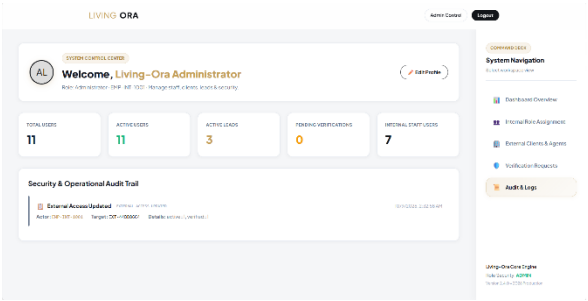
Admin UI



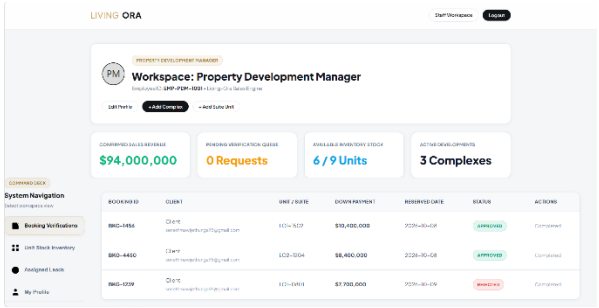


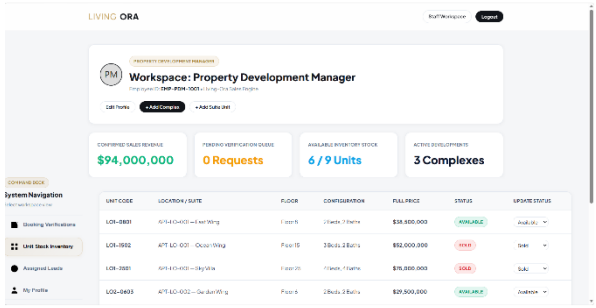


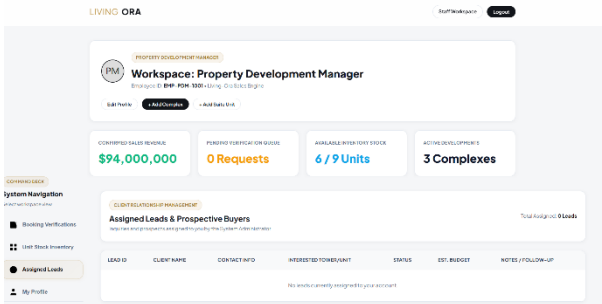


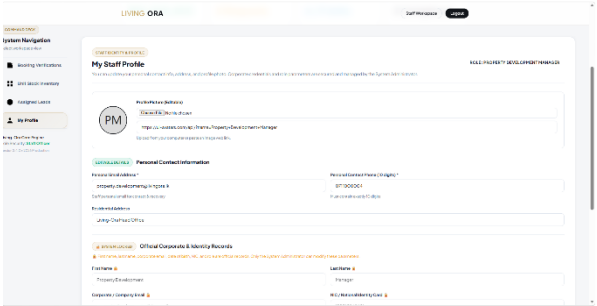


Staff UI









3.4 Design Patterns Applied

The Living-Ora Apartment Sales System applies the Strategy pattern (GoF behavioral) to Booking & Reservation Management. It separates apartment availability operations into interchangeable strategies selected according to the requested booking status. The booking module also uses layered architecture, repositories, DTOs, and Spring framework mechanisms.

Pattern / Mechanism	Type	Where applied	Why it was chosen
Strategy	GoF – Behavioural	BookingStatusStrategy, ApproveBookingStrategy, ReleaseBookingStrategy, PendingBookingStrategy; used by BookingService	Separates status-specific availability operations and allows the service to delegate to the appropriate algorithm at runtime.
Strategy	GoF – Behavioural	PaymentProcessingStrategy (interface), BankTransferPaymentStrategy, OnlineCardPaymentStrategy, OtherPaymentStrategy, PaymentStrategyResolver; used by PaymentService	Separates payment methods and uses PaymentStrategyResolver to select the appropriate strategy at runtime, making it easy to add new payment methods.
Layered Architecture / Spring MVC	Architectural	BookingController → BookingService → repositories; entities and DTOs represent data	Separates HTTP request handling, business logic, and persistence.
Repository	Data access pattern; non-GoF	BookingRepository, UnitRepository, PaymentRepository	Encapsulates database access behind interfaces, reducing persistence code in services.
DTO / Mapper	Data transfer and mapping patterns; non-GoF	BookingRequest, BookingUpdateRequest, BookingResponse, BookingMapper	Separates API input/output from persistence entities and centralises conversion between them.
Dependency Injection	Spring framework mechanism	@Autowired in the controller and service; constructor injection in strategies and the registry	Supplies dependencies without callers constructing concrete collaborators, supporting loose coupling and testing.
Singleton Scope	Spring lifecycle mechanism	Spring-managed services, repositories, and strategy components use singleton scope by default	Reuses one bean instance per Spring container. This is distinct from explicitly implementing the GoF Singleton pattern.
Declarative Transactions	Spring framework mechanism	@Transactional on booking service operations	Groups related booking, payment, and apartment updates into a transaction.

Strategy Pattern – Booking Status Handling

Problem – Different booking status updates require different apartment availability operations. Previously, BookingService.updateBookingStatus() contained an if/else chain that marked apartments as sold or available. Keeping these operations in the service would make it grow whenever another availability algorithm was introduced.

Solution – The operations are extracted into classes implementing BookingStatusStrategy. Each strategy declares its supported statuses through supportedStatuses() and performs its operation through apply(Booking booking).

Concrete strategy	Supported booking status	Behaviors
ApproveBookingStrategy	Approved	Changes the apartment's availability to Sold.
ReleaseBookingStrategy	Rejected, Cancelled, Expired	Changes the apartment's availability to Available.
PendingBookingStrategy	Pending Approval	Leaves the apartment's availability unchanged.

BookingStatusStrategyRegistry receives the strategy objects through Spring dependency injection and registers their supported statuses. BookingService resolves the appropriate strategy, validates the existing booking, updates its status, and invokes strategy.apply(booking).
Where it is used. BookingController.updateBookingStatus() handles requests to PATCH /api/bookings/{id}/status and calls BookingService.updateBookingStatus(). For example, a cancellation selects ReleaseBookingStrategy, which makes the apartment available again. The service then saves the booking within the same transaction.

Benefits – The design separates availability algorithms from booking coordination, supports independent testing, and allows additional strategies to be registered without adding availability-specific branches to the service. Existing transition rules such as preventing changes to an approved booking remain the service's responsibility.

Strategy Pattern – Payment Handling

Problem – Different payment methods require different payment-processing operations. Implementing all payment methods within PaymentService would create conditional logic and make the service more difficult to maintain whenever a new payment method is introduced.
Solution – The payment-processing operations are separated into classes implementing the PaymentProcessingStrategy interface. Each concrete strategy handles a specific payment method, while PaymentStrategyResolver selects the appropriate strategy at runtime.

Concrete strategy	Supported booking status	Behaviors
`BankTransferPaymentStrategy`	Bank Transfer	Handles payment processing for bank transfers.
`OnlineCardPaymentStrategy`	Online Card Payment	Handles payment processing for online card payments.
`OtherPaymentStrategy`	Other payment methods	Handles payment methods not covered by the other concrete strategies.

PaymentStrategyResolver selects the appropriate payment strategy through Spring dependency injection, and PaymentService delegates the processing operation to it.
Where it is used PaymentService uses this pattern to handle different payment methods, such as bank transfers, through their respective strategies.

Benefits – The design separates payment-processing algorithms from service coordination, improves maintainability, and supports independent testing. New payment methods can be introduced by implementing PaymentProcessingStrategy and registering the new strategy, reducing the need to modify existing service logic.

4. Implementation

4.1 Tools & Technologies Used

Area	Technology
Language / runtime	Java 17
Framework	Spring Boot 4.1.1
Web	Spring Web MVC
Persistence	Spring Data JPA, JDBC
Validation	Spring Boot Validation
Security	Spring Security Crypto
Database	MySQL
Test Database	H2
HTTP Client	Spring REST Client
Build Tool	Gradle with Kotlin DSL
Testing	Spring Boot JDBC Test, REST Client Test, Web MVC Test, JUnit Platform
Build Tool	Gradle with Kotlin DSL
Dependency Management	Spring Dependency Management Plugin 1.1.7
IDE / VCS	IntelliJ IDEA, Git

4.2 System Architecture

The application follows a layered architecture under the package com. apartmentsalessystembackend. The project is organized into separate packages so that responsibilities are clearly divided between configuration, request handling, data transfer, persistence, business logic, exception handling, and utility functions.

- **config**: Contains application-level configuration classes such as CorsConfig, DataInitializer, JpaConfig, and SecurityConfig. These classes configure cross-origin access, database initialization, JPA behavior, and application security.
- **controller**: Acts as the presentation/API layer of the system. Controllers handle incoming HTTP requests, call the appropriate service-layer functions, and return responses to the client.
- **dto (request / response)**: Contains Data Transfer Objects used to exchange data between the client and backend. Request DTOs carry incoming data, while response DTOs return processed information without directly exposing entity objects.
- **entity**: Contains the JPA entity classes that represent the main domain objects of the Apartment Sales System and are mapped to the relational database.
- **exception**: Provides centralized exception handling. This package includes custom exceptions such as BadRequestException, ForbiddenException, and ResourceNotFoundException, together with GlobalExceptionHandler for managing application errors consistently.
- **mapper**: Responsible for converting data between entity objects and DTOs. This helps keep persistence-layer objects separate from the data exposed through the API.
- **repository**: Contains Spring Data JPA repository interfaces used for database operations such as creating, retrieving, updating, and deleting records.
- **service**: Contains the main business logic of the application. Services coordinate operations between controllers, repositories, validation utilities, and other application components.

- **util**: Contains reusable utility classes such as JwtUtil, PasswordHasher, and ValidationUtil. These provide support for JWT-based authentication, password hashing, and input validation.

The architecture maintains separation of concerns by ensuring that controllers handle HTTP communication, services contain business logic, repositories manage database access, and DTOs are used for data exchange. Security-related operations are handled through SecurityConfig, JwtUtil, and PasswordHasher, while validation and exception handling are centralized to improve maintainability and consistency.

A suitable architectural flow for the system is:
Client → Controller → Service → Repository → MySQL Database

For authenticated operations, the request also passes through security configuration and JWT-related components before accessing protected resources.

4.3 Key Features Developed

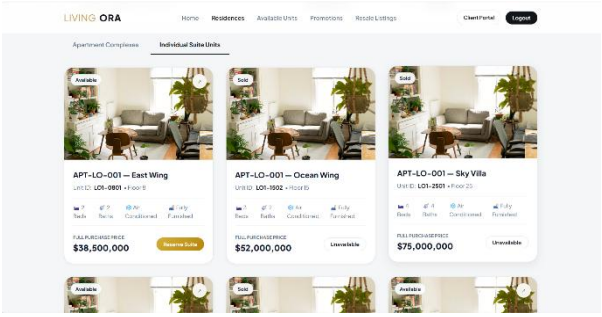
Features	Description
Lead Management	Capture, organize, assign, update, and follow customer leads through the sales pipeline while retaining relevant follow-up information.
Apartment Inventory Management	Maintain apartment project and unit information including pricing, specifications, availability, and status.
Booking and Reservation Management	Allow eligible users to reserve available units, manage booking details, and prevent duplicate allocation.
Payment and Installment Tracking	Record payments, maintain installment schedules, track balances and due dates, and generate payment records or receipts.
Promotions and Special Offers Management	Create, update, activate/deactivate, and present promotional offers to relevant users.
Agent Portfolio and Performance Management	Track assigned external Apartments, sales activity, performance, and target-related information for sales personnel.

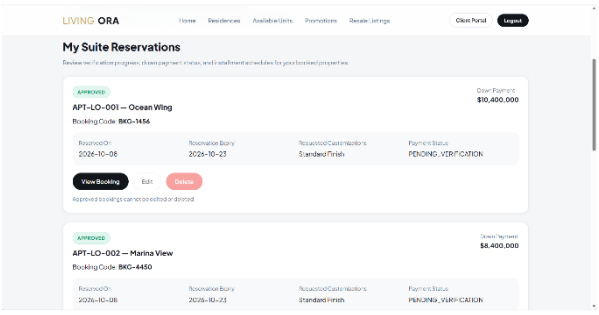
Validation, Data Integrity and Security
The system also implements supporting mechanisms for validation, data integrity, and security. Input validation is applied to reduce incomplete or invalid data, while business rules are used to prevent inappropriate operations such as invalid status changes or reserving unavailable apartment units. The booking workflow updates unit availability to reduce duplicate allocation.

The backend also separates security-related responsibilities through components such as SecurityConfig, JwtUtil, PasswordHasher, and validation utilities. Role-based access control is used to restrict protected system operations to appropriate users.

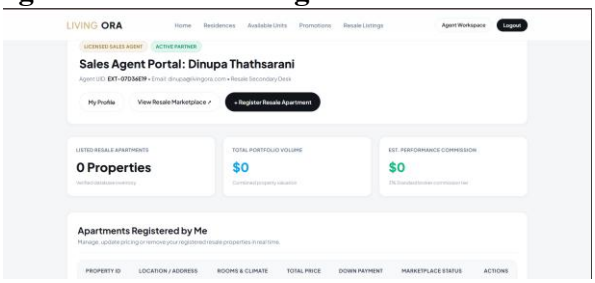
4.4 Screenshots of Key Functions

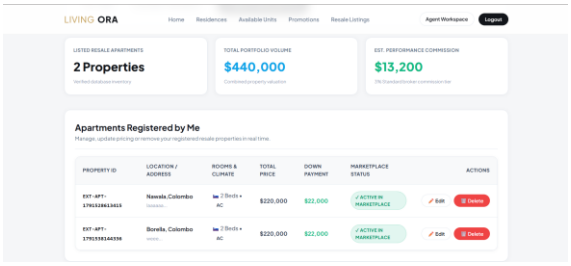
Booking & Reservation Management



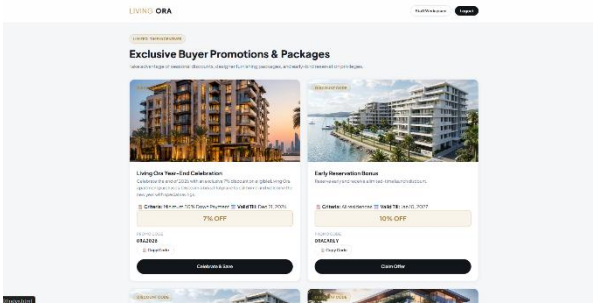


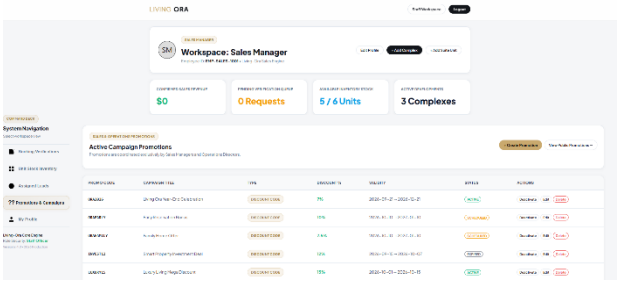
Agent Portfolio Management



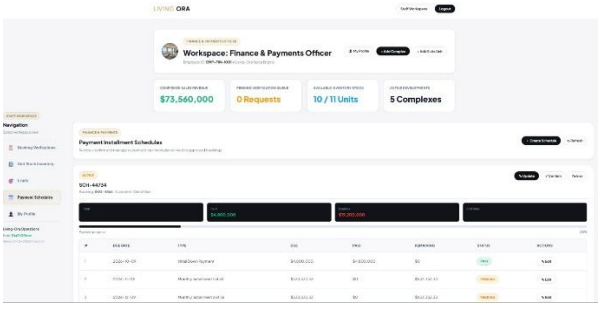


Promotions & Special Offers Management

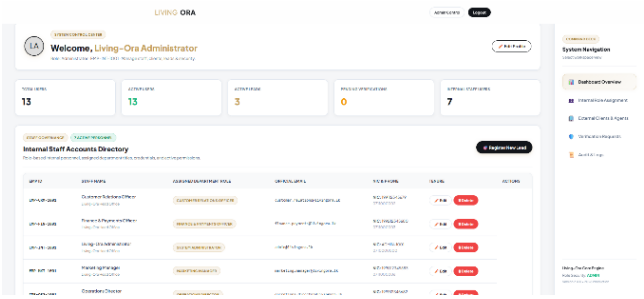


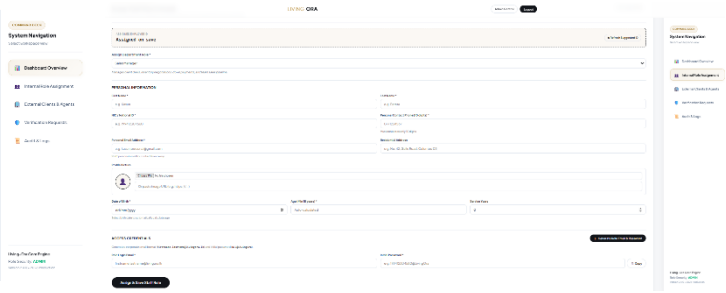


Payment & Installment Schedule Tracking

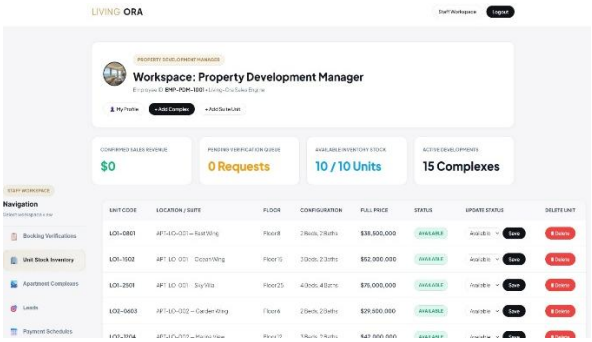


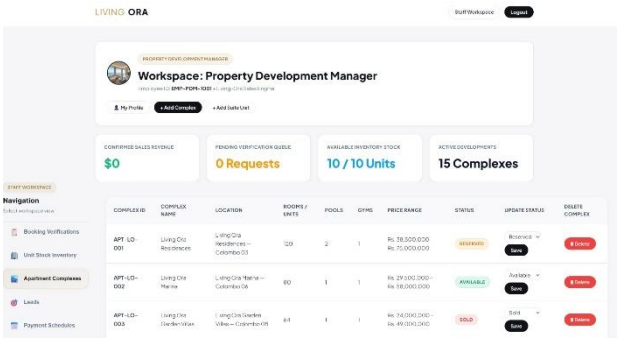
Lead Management





Apartment Inventory Management





5. Project Management

5.1 Agile Approach

The project follows Scrum. The corrected Design Document records a sequence from initiation and requirements through architecture, shared user management, major-module development, integration/testing and final review. Earlier Scrum artefacts also define a Product Owner, Scrum Master and development focuses for UI/UX, backend, frontend and QA, supported by a prioritized product backlog and estimated sprint tasks.

Scrum Role	Member	Responsibility
Product Owner	Jayasuriya J.A.P.S	Prioritizes backlog, confirms acceptance criteria and represents overall project requirements.
Scrum Master	Mannapperuma M.J.P	Facilitates Scrum activities, supports coordination, tracks blockers and helps the team follow the process.
Developer - UI/UX focus	Bandara A.M.P.L	Designs interfaces, forms, dashboards and interaction flows.
Developer - Backend focus	Bandara R.H.D.T	Implements APIs, business logic, validation rules and data operations.
Developer - Frontend focus	Shankeerthan S	Implements user-facing screens and integrates frontend workflows with services.
Developer - QA focus	Wijethunga S.M	Validates completed features, performs workflow testing and records defects.

5.2 Final Sprint Summary

Sprint	Period	Status in Corrected Design	Key Deliverable
Sprint 0	Weeks 3-4	Completed	Project initiation, scope definition, requirements and prioritized backlog.
Sprint 1	Week 5	Completed	System analysis, architecture, EER/class foundation and initial use-case structure.
Sprint 2	Weeks 6-7	Completed	Shared registration/login, role dashboards/navigation and account validation.
Sprint 3	Week 7	Completed	Lead Management and Agent Portfolio Management.
Sprint 4	Weeks 8-9	Completed	Apartment Inventory and Promotions & Special Offers.
Sprint 5	Week 10	Completed	Booking & Reservation, including unit-status safeguards.
Sprint 6	Weeks 10-11	Completed	Payment/installment tracking and financial reporting.
Sprint 7	Week 12	Completed	System integration, end-to-end testing, consistency and edge-case handling.
Sprint 8	Weeks 12-13	Completed	Final report, updated diagrams, consolidated fixes and viva preparation.

5.3 Task Distribution

Member	Student ID	Assigned Major Function
Jayasuriya J.A.P.S	IT25101419	Payment & Installment Schedule Tracking
Mannapperuma M.J.P	IT25102211	Apartment Inventory Management
Bandara A.M.P.L	IT25103164	Lead Management
Bandara R.H.D.T	IT25100456	Agent Portfolio Management
Wijethunga S.M	IT25102571	Booking & Reservation Management
Shankeerthan S	IT25100040	Promotions & Special Offers Management

5.4 Development Milestones

The project lifecycle progressed from planning and requirements into architecture and shared user management before the six major functions were developed. Integration and final documentation follow the module-development sprints. This sequencing reduces cross-module risk because common authentication and data structures are established before the business modules depend on them.

6. Conclusion & Future Works

6.1 Summary of Achievements

The Web-based Apartment Sales System consolidates the core information flows of apartment sales into a structured web solution. The project defines and develops role-based access, customer/lead handling, apartment inventory, promotions, booking/reservation, payment/installment tracking, agent portfolio functions and management reporting. The corrected design integrates these functions through final use-case and class models, while the EER model provides the relational-data foundation.

A particularly important design achievement is the connection between booking and apartment status. By checking current availability and updating the unit state at reservation or sale, the workflow is designed to reduce double allocation. The project also recognizes the need for financial transparency through payment history, installment plans, due dates and outstanding-balance reporting, and for management visibility through dashboards and performance reports.

6.2 Key Challenges Addressed

The supplied artefacts identify technical and project risks more clearly than they document personal difficulties. The main system-level challenges that the team needed to manage are therefore summarized below without inventing individual experiences:

- Maintaining consistency across interconnected modules when a reservation, promotion, payment or unit status changes.
- Preventing conflicting bookings and invalid unit-status transitions.
- Ensuring role-based access protects customers and financial information.
- Integrating independently developed modules into a stable end-to-end workflow.
- Providing validation and useful error handling while keeping the interface easy to use.
- Completing integration testing, documentation and viva preparation within the academic schedule.

6.3 Future Improvements

Improvement	Future Direction
Advanced notifications	Add verified email/SMS/push integrations for reservation confirmations, payment reminders and promotion alerts.
Mobile application	Introduce a dedicated mobile client if future scope and resources justify it.
Enterprise scalability	Extend the architecture for multi-project or multi-organization operation with stronger deployment and monitoring capabilities.
Analytics	Expand dashboards with deeper trend analysis, conversion funnels and forecasting once sufficient real data is available.
Integration	Connect appropriate payment gateways and external property/CRM services after security, legal and data-governance review.
Accessibility and usability	Conduct structured accessibility and usability testing across devices and user roles.
Automated testing	Increase unit, integration and end-to-end test coverage for critical booking, payment and status-transition workflows.

7. Contribution & Lessons Learned

The assignment requires one reflection row per student. The documented roles and major-function ownership below come directly from the project artefacts. The reflective wording is a draft prepared from those documented responsibilities because the supplied files do not contain each member's personal reflection. Each member should read and confirm their row before submission so that it accurately represents their own experience and can be defended in the viva.

Member	Documented Role / Major Function	Specific Contribution	Challenge / Risk	How Addressed	Lessons Learned
Jayasuriya J.A.P.S IT25101419	Product Owner; Payment & Installment Schedule Tracking	Worked with payment records, installment schedules and related financial reporting while also prioritizing backlog/acceptance criteria.	The function required careful consistency between bookings, payments, balances and due dates.	Used clear acceptance criteria and structured payment records to keep finance behaviour aligned with the sales workflow.	Learned the importance of backlog prioritization, data consistency and explaining financial workflows clearly.
Mannapperuma M.J.P IT25102211	Scrum Master; Apartment Inventory Management	Coordinated Scrum activities and contributed to project/unit inventory, pricing and availability-related functions.	Inventory data is shared by buyer search, booking, promotions and management reporting, so inconsistency can affect several modules.	Used coordinated sprint planning and a centralized inventory/status model to support cross-module use.	Improved understanding of Scrum coordination, dependency management and maintaining a reliable source of inventory truth.
Bandara A.M.P.L IT25103164	UI/UX focus; Lead Management	Designed user interaction flows and contributed to lead capture and pipeline tracking.	Lead screens must capture enough information for follow-up without making the workflow difficult for users.	Applied structured forms, validation and clear lead-status flows to support usability and completeness.	Learned how UI decisions influence data quality and how user stories can be translated into practical screens and workflows.
Bandara R.H.D.T IT25100456	Backend focus; Agent Portfolio Management	Focused on APIs/business logic/data operations and contributed to agent portfolio and performance functionality.	Backend rules need to keep assignments, agent activity and performance information consistent across modules.	Used explicit business logic and validation boundaries so portfolio data can be consumed reliably by reporting views.	Strengthened understanding of service-layer logic, validation, data relationships and integration with frontend workflows.
Wijethunga S.M IT25102571	QA focus; Booking & Reservation Management	Contributed to reservation workflows and QA activities, including validation of completed features and end-to-end workflow behaviour.	A critical risk is duplicate booking or invalid status changes when multiple operations affect the same apartment unit.	Focused on availability checks, status-transition validation and workflow testing around reservation outcomes.	Learned the value of testing edge cases, validating business rules and tracing how one user action changes data used elsewhere.
Shankeerthan S IT25100040	Frontend focus; Promotions & Special Offers Management	Implemented user-facing workflow integration and contributed to promotion/special-offer management.	Promotion information must be understandable to buyers and must respect dates, conditions and linked apartment information.	Used clear promotion states/conditions and connected frontend behaviour to validated backend data.	Improved skills in frontend-service integration, presenting business rules to users and maintaining consistent interaction patterns.

The project structure distributed responsibilities while requiring repeated integration across functional boundaries. Scrum roles provided coordination responsibilities in addition to the six major functions. The backlog/task model also assigned requirements, UI/UX, backend, frontend and QA work across members, encouraging shared responsibility rather than isolated module ownership. For the final viva, every member should be able to explain both their own major function and how it interacts with authentication, the database, validation and at least one neighboring module.

8. References

[1] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum Guides, Nov. 2020. [Online]. Available: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf.

[2] Spring, *Spring Boot Reference Documentation*. [Online]. Available: https://docs.spring.io/spring-boot/reference/.

[3] Spring, *Spring Data JPA Reference Documentation*. [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/.

[4] Spring, *Spring Security Reference*. [Online]. Available: https://docs.spring.io/spring-security/reference/.

[5] Gradle Inc., *Gradle Kotlin DSL Primer*. [Online]. Available: https://docs.gradle.org/current/userguide/kotlin_dsl.html.

[6] Oracle, *Java Platform, Standard Edition 17 Documentation*. [Online]. Available: https://docs.oracle.com/en/java/javase/17/.

[7] Oracle, *MySQL 8.4 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/.

[8] H2 Database Engine, *H2 Database Documentation*. [Online]. Available: https://www.h2database.com/html/main.html.

[9] OWASP Foundation, *Application Security Verification Standard*. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/.

[10] OWASP Foundation, *Password Storage Cheat Sheet*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.

[11] OWASP Foundation, *JSON Web Token for Java Cheat Sheet*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html.